

Explicacion delCodigo

Sistema Municipal de Turnos - Java, Swing, SOLID/LSP y Supabase

Este documento explica el proyecto archivo por archivo y metodo por metodo, con foco en como se conectan la interfaz grafica, la logica de negocio, el principio de Liskov y la persistencia en Supabase.

1. Vista general de la arquitectura

El proyecto esta dividido en capas simples para que cada parte tenga una responsabilidad clara.

Capa	Archivos	Responsabilidad
Entrada	Main.java	Arranca la aplicacion Swing y conecta dependencias.
Interfaz grafica	SistemaTurnosFrame.java	Pantallas, botones, tabla y mensajes al usuario.
Logica de negocio	GestorTurnos.java	Crea turnos, calcula fechas, costos y descuentos.
Acceso a datos	SupabaseClient.java	Hace GET, POST y PATCH contra Supabase REST API.
Modelo SOLID	TurnoMunicipal, TurnoArancelado, TurnoRepresentante, TurnoSocio	Representa los tipos de turno y demuestra Liskov.
DTOs	SolicitudTurno, ResultadoTurno, Ciudadano, CiudadanoTurno, CiudadanoDetalle	Transporta datos entre capas.

Flujo general

- El usuario usa la ventana Swing.
- La ventana arma una SolicitudTurno con los datos ingresados.
- GestorTurnos decide que tipo de turno crear, calcula fecha, monto y descuento.
- SupabaseClient guarda o consulta datos en Supabase.
- La interfaz muestra el resultado en pantalla o en una tabla.

2. Main.java

Este archivo quedo intencionalmente chico. Su tarea es iniciar el programa y no contener reglas de negocio. Esto responde a la devolucion de descomprimir Main.

main(String[] args)

Punto de entrada de la aplicacion.

- Usa SwingUtilities.invokeLater para iniciar Swing en el hilo correcto de interfaz grafica.
- Configura el Look and Feel del sistema operativo.
- Crea SupabaseClient, que sabe comunicarse con la base.
- Crea GestorTurnos, que contiene la logica de negocio.
- Crea SistemaTurnosFrame y lo muestra.
- Si falta una variable de entorno o ocurre otro error al iniciar, muestra un JOptionPane con el problema.

```
SupabaseClient supabaseClient = new SupabaseClient();
GestorTurnos gestorTurnos = new GestorTurnos(supabaseClient);
new SistemaTurnosFrame(gestorTurnos).setVisible(true);
```

3. Interfaces del modelo

Estas interfaces son la base de la aplicacion del principio de Sustitucion de Liskov.

TurnoMunicipal.getDetalleTurno()

Obliga a cualquier turno municipal a devolver una descripcion legible del tramite.

TurnoMunicipal.getFechaHora()

Obliga a cualquier turno a exponer la fecha y hora asignada.

TurnoMunicipal.confirmarTurno()

Define una accion comun: confirmar el turno. Cada tipo lo implementa con su mensaje propio.

TurnoArancelado.calcularCosto()

Solo existe para turnos pagos. No se fuerza a TurnoSocial a tener costo.

TurnoArancelado.generarBoletoPago()

Representa el comportamiento especifico de un tramite arancelado.

Punto clave: TurnoArancelado extiende TurnoMunicipal. Por eso todo turno arancelado tambien es municipal, pero no todo turno municipal es arancelado.

4. Modelos: TurnoLicencia y TurnoSocial

TurnoLicencia(String fechaHora, String categoriaLicencia)

Constructor del turno de licencia. Guarda fecha/hora y categoria de licencia.

TurnoLicencia.getDetalleTurno()

Devuelve un texto como Renovacion Licencia Categ. B1.

TurnoLicencia.getFechaHora()

Devuelve la fecha asignada por el sistema.

TurnoLicencia.confirmarTurno()

Muestra que el turno de licencia fue confirmado y queda pendiente de pago.

TurnoLicencia.calcularCosto()

Calcula el costo del tramite. Para B1 devuelve 15000. Para otras categorias devuelve 20000.

TurnoLicencia.generarBoletoPago()

Muestra en consola un mensaje de boleto por el costo calculado.

TurnoSocial(String fechaHora, String motivo)

Constructor del turno social. Guarda fecha/hora y motivo.

TurnoSocial.getDetalleTurno()

Devuelve un texto indicando que es un turno social gratuito y su motivo.

TurnoSocial.getFechaHora()

Devuelve la fecha asignada por el sistema.

TurnoSocial.confirmarTurno()

Muestra que el turno social fue confirmado.

Relacion con Liskov: GestorTurnos puede trabajar con una variable TurnoMunicipal y asignarle un TurnoLicencia o un TurnoSocial sin romper el contrato comun.

5. DTOs: SolicitudTurno y ResultadoTurno

Los records son contenedores de datos. Reducen código repetitivo porque Java genera constructor, getters, equals, hashCode y toString.

SolicitudTurno(...)

Representa los datos que entran desde la interfaz cuando el usuario quiere sacar un turno.

- dni: identifica al ciudadano.
- nombreCompleto, email, telefono: datos personales.
- mayor65: indica si corresponde descuento.
- servicio: nombre del servicio elegido.

ResultadoTurno(...)

Representa lo que vuelve después de guardar un turno.

- idTurno: ID creado en Supabase.
- detalle y fechaHora: información del turno creado.
- arancelado: indica si genero pago.
- montoOriginal, descuentoAplicado, montoFinal: resumen económico del trámite.

6. GestorTurnos.java

Esta clase concentra la lógica de negocio. La GUI no calcula fechas ni descuentos; se lo pide a GestorTurnos.

GestorTurnos(SupabaseClient supabaseClient)

Constructor. Recibe el cliente de Supabase por parámetro para poder persistir datos.

sacarTurno(SolicitudTurno solicitud)

Método principal para crear un turno.

- Crea un TurnoMunicipal concreto con crearTurnoMunicipal.
- Calcula el monto original si el turno es arancelado.
- Calcula descuento si el ciudadano es mayor de 65 y el trámite es pago.
- Calcula monto final.
- Llama a SupabaseClient.registrarTurno para guardar ciudadano, turno y pago.
- Devuelve ResultadoTurno para que la GUI pueda mostrar un resumen.

obtenerTurnosPorDni(String dni)

Delega en SupabaseClient la consulta de turnos por DNI.

buscarCiudadanoPorDni(String dni)

Delega en SupabaseClient la búsqueda de ciudadano para reutilizar datos existentes.

pagarTurnoArancelado(String dni, int idTurno)

Delega en SupabaseClient la operación de marcar un turno pago.

cancelarTurno(String dni, int idTurno)

Delega en SupabaseClient la cancelación lógica del turno.

obtenerServiciosDisponibles()

Devuelve la lista de servicios que la GUI carga en el combo.

crearTurnoMunicipal(String servicio)

Decide que clase instanciar segun el servicio.

- Si el servicio es Licencia de conducir, crea TurnoLicencia.
- Si el servicio es Asistencia social alimentaria, crea TurnoSocial.
- Si llega otro texto, lanza error de servicio no reconocido.

asignarFechaHoraTurno(String servicio)

Asigna fechas automaticamente.

- Licencia: fecha actual + 7 dias a las 09:00.
- Social: fecha actual + 3 dias a las 10:30.
- Devuelve el formato yyyy-MM-dd HH:mm:ss para Supabase.

calcularMontoOriginal(TurnoMunicipal turno)

Si el turno implementa TurnoArancelado, llama calcularCosto. Si no, devuelve 0.

calcularDescuento(TurnoMunicipal turno, boolean mayor65, double montoOriginal)

Aplica 30% de descuento solo si el ciudadano es mayor de 65 y el turno es arancelado.

7. SistemaTurnosFrame.java

Es la ventana principal Swing. Contiene tres pestañas: Sacar turno, Mis turnos y Servicios. No debería contener reglas profundas de negocio; usa GestorTurnos para eso.

SistemaTurnosFrame(GestorTurnos gestorTurnos)

Constructor de la ventana.

- Guarda la referencia al gestor.
- Configura cierre de ventana, tamaño y posición.
- Inicializa componentes.
- Carga el contenido principal con pestañas.

configurarComponentes()

Carga servicios en el JComboBox, bloquea áreas de salida para que no sean editables y habilita ordenamiento en la tabla.

crearContenido()

Crea el JTabbedPane con las pestañas Sacar turno, Mis turnos y Servicios.

crearPanelAlta()

Arma la pestaña para sacar turno.

- Campos: DNI, nombre, email, teléfono, mayor de 65, servicio.
- Botón Buscar ciudadano.
- Botón Sacar turno.
- Botón Limpiar formulario.
- Área de salida con resumen del turno creado.

crearPanelConsulta()

Arma la pestaña para consultar, pagar y cancelar turnos.

- Campo DNI.
- Tabla de turnos.
- Botones Ver mis turnos, Marcar pagado y Cancelar turno.

crearPanelServicios()

Arma una pestaña informativa con los servicios disponibles y sus condiciones.

addField(...)

Método auxiliar para agregar una etiqueta y un campo al formulario usando GridBagLayout.

buscarCiudadano()

Busca el ciudadano por DNI.

- Valida que el DNI no esté vacío.
- Llama a gestorTurnos.buscarCiudadanoPorDni en segundo plano.
- Si existe, completa campos y los bloquea para no duplicar datos.
- Si no existe, desbloquea campos para crear el ciudadano.

sacarTurno()

Lee datos del formulario y crea una `SolicitudTurno`.

- Valida DNI y nombre.
- Llama a `gestorTurnos.sacarTurno`.
- Muestra el `ResultadoTurno` formateado.

cargarTurnos()

Consulta turnos por DNI y los carga en la tabla.

pagarTurnoSeleccionado()

Obtiene el ID de la fila seleccionada y marca el turno arancelado como pagado.

cancelarTurnoSeleccionado()

Obtiene el ID de la fila seleccionada y marca el turno como Cancelado.

obtenerIdSeleccionado()

Convierte la fila seleccionada de la `JTable` al índice real del modelo y devuelve el ID de turno.

mostrarTurnos(List turnos)

Limpia la tabla y agrega una fila por cada turno con datos de servicio, estado, descuento y pago.

formatearResultado(ResultadoTurno resultado)

Construye el texto que se muestra al crear un turno. Si es arancelado muestra monto, descuento y total.

limpiarAlta()

Limpia el formulario de alta y vuelve a habilitar los campos.

bloquearDatosCiudadano()

Deshabilita edicion de datos cuando el ciudadano ya existe.

desbloquearDatosCiudadano()

Habilita edicion cuando el ciudadano no existe o se limpia el formulario.

ejecutarAsync(Operacion operacion)

Ejecuta operaciones lentas en un `Thread` aparte para no congelar la interfaz `Swing`.

mostrarError(String mensaje)

Muestra errores en un `JOptionPane`.

Operacion

Interfaz funcional interna usada por `ejecutarAsync` para recibir acciones que pueden lanzar excepciones.

8. SupabaseClient.java

Es la capa de acceso a datos. Construye peticiones HTTP para Supabase REST. No tiene pantallas ni reglas visuales.

readRequiredEnv(String name)

Lee variables de entorno obligatorias.

- SUPABASE_URL debe tener la URL base, sin /rest/v1.
- SUPABASE_SERVICE_ROLE_KEY debe tener la secret key.
- Si falta una variable, lanza IllegalStateException.
- Si la URL termina en /, lo quita para evitar doble barra.

registrarTurno(...) version antigua

Metodo de compatibilidad. Llama a la version nueva con mayor65 false y montos 0.

registrarTurno(...) version nueva

Guarda un turno completo.

- Busca o crea ciudadano.
- Busca o crea servicio.
- Crea turno.
- Si el servicio es arancelado, crea pago con monto original, descuento y monto final.
- Devuelve el ID del turno creado.

obtenerTurnosPorDni(String dni)

Consulta turnos de un ciudadano usando relaciones REST.

- Consulta turnos.
- Incluye servicios, pagos y ciudadanos mediante select relacional.
- Filtra por ciudadanos.dni.
- Ordena por fecha_hora descendente.
- Convierte JSON a lista de TurnoDetalle.

buscarCiudadanoPorDni(String dni)

Consulta datos del ciudadano por DNI. Devuelve null si no existe.

pagarTurnoArancelado(String dni, int idTurno)

Marca como pagado sin eliminar.

- Primero trae los turnos del DNI.
- Verifica que el ID pertenezca al ciudadano.
- Verifica que sea arancelado.
- Actualiza pagos.estado_pago a Pagado y fecha_pago a la fecha actual.
- Actualiza turnos.estado a Pagado.

cancelarTurno(String dni, int idTurno)

Valida que el turno pertenezca al DNI y actualiza turnos.estado a Cancelado.

buscarOCrearCiudadano(...)

Busca ciudadano por DNI. Si existe devuelve su ID. Si no existe, arma JSON y lo inserta en ciudadanos.

buscarServicio(String nombreServicio)

Busca el servicio por nombre. Si no existe, llama a crearServicio.

crearServicio(String nombreServicio)

Crea Licencia de conducir o Asistencia social alimentaria con sus valores base.

crearTurno(...)

Inserta en turnos id_ciudadano, id_servicio, fecha_hora y estado Pendiente. Devuelve id_turno.

crearPago(...)

Inserta en pagos id_turno, monto_original, descuento_aplicado, monto_final y estado_pago Pendiente.

get(String path) / get(String path, String dni)

Construye y envia una peticion HTTP GET.

post(String path, String body, String dni)

Construye y envia una peticion HTTP POST con JSON. Usa Prefer: return=representation para que Supabase devuelva el registro creado.

patch(String path, String body, String dni)

Construye y envia una peticion HTTP PATCH para actualizar registros sin borrarlos.

requestBuilder(String path, String dni)

Centraliza headers y URL.

- Agrega apikey con la secret key.
- Si la key es legacy JWT empieza con eyJ y agrega Authorization Bearer.
- Si se pasa DNI, agrega header x-dni.

send(HttpRequest request)

Ejecuta la peticion HTTP. Si el status no es 2xx, lanza IOException con el cuerpo del error.

encode(String value)

Codifica valores para URL, reemplazando espacios por %20.

escapeJson(String value)

Escapa barras y comillas para evitar romper el JSON construido manualmente.

extractObjects(String jsonArray)

Separa objetos dentro de una respuesta JSON tipo array.

extractInt / extractNullableInt

Extraen enteros desde JSON. La version nullable devuelve null si no encuentra el campo.

extractString / extractNullableString

Extraen textos desde JSON.

extractBoolean

Extrae booleanos true/false desde JSON.

extractDouble / extractNullableDouble

Extraen numeros decimales desde JSON.

ServicioMunicipal

Record privado para manejar internamente datos de la tabla servicios.

TurnoDetalle

Record publico que representa una fila de turno ya lista para mostrar en la GUI.

Ciudadano

Record publico que representa un ciudadano encontrado por DNI.

9. ProcesadorTurnos.java

Quedo como servicio conceptual de apoyo para demostrar polimorfismo y Liskov por consola, aunque el flujo principal actual usa Swing y GestorTurnos.

imprimirReporteTurnos(List turnos)

Recibe cualquier lista de TurnoMunicipal. Puede imprimir TurnoSocial o TurnoLicencia sin distinguir clases concretas.

procesarPagos(List turnosArancelados)

Recibe solamente turnos arancelados, por eso puede llamar generarBoletoPago con seguridad.

10. Como se aplica Liskov en este codigo

El sistema respeta Liskov porque separa el contrato general de los turnos del contrato especifico de los pagos.

- TurnoMunicipal define solo lo que todo turno debe cumplir.
- TurnoSocial implementa TurnoMunicipal porque es gratuito y no necesita costo.
- TurnoLicencia implementa TurnoArancelado, que a su vez extiende TurnoMunicipal.
- GestorTurnos puede guardar un TurnoLicencia o TurnoSocial dentro de una variable TurnoMunicipal.
- El costo se calcula solo si el objeto tambien es TurnoArancelado.

```
TurnoMunicipal turno = new TurnoLicencia(fechaHora, "B1");
// o tambien:
TurnoMunicipal turno = new TurnoSocial(fechaHora, "Asistencia alimentaria");

// En ambos casos esto funciona:
turno.getDetalleTurno();
turno.getFechaHora();

// Pero calcularCosto solo se usa si es TurnoArancelado.
```

11. Configuracion necesaria

Para ejecutar la aplicacion, IntelliJ o PowerShell deben tener estas variables de entorno:

```
SUPABASE_URL=https://vqbdyvcmyvcyewkxcek.supabase.co
SUPABASE_SERVICE_ROLE_KEY=sb_secret_...
```

No se debe subir la secret key al repositorio. El proyecto ya esta preparado para leerla desde variables de entorno.